Cloudstock: AI Research Assistant on AWS

Executive Summary

Cloudstock is a simple cloud-based AI research assistant that allows users to input a company name and receive a structured analysis including a summary, risks, and potential opportunities. The goal of the project was to design and deploy a working application using AWS services while following basic cloud architecture principles.
The application is deployed using AWS Elastic Beanstalk as the primary compute service, with Amazon EC2 running the application behind the scenes. Amazon S3 is used for storage, and Amazon CloudWatch is used for logging and monitoring. The project demonstrates how a basic application can be deployed, managed, and observed in a cloud environment with minimal infrastructure overhead.

Architecture Design

The system follows a straightforward architecture:

- The user accesses the application through a public URL.
- The request is routed to an Elastic Beanstalk environment.
- Elastic Beanstalk manages an EC2 instance that runs the Flask application using Gunicorn.
- The application processes the input and returns a generated analysis.
- Logs and application activity are sent to CloudWatch.
- S3 is used for storing project-related assets.

Each component has a clear role:

- Elastic Beanstalk handles deployment, scaling, and environment management. It removes the need to manually configure servers.
- EC2 runs the actual application code.
- S3 provides simple and scalable storage.
- CloudWatch captures logs and helps debug issues during deployment and runtime.
- IAM roles are used to securely allow services to interact without exposing credentials.

This architecture is intentionally simple but reflects how real applications are structured in AWS.

Implementation Details

The application was built using Python and Flask. It accepts a company name as input and generates a basic analysis using a deterministic function.

Setup Process

1. The Flask application was created locally with:
    - application.py
    - requirements.txt

    - Procfile
    - templates/index.html
2. Dependencies were defined in requirements.txt, including Flask and Gunicorn.
3. The application was packaged into a ZIP file and deployed through the Elastic Beanstalk console.
4. Once deployed, the application was accessible through a public AWS URL.

Challenges & Solutions

The main issue encountered was a 502 Bad Gateway error after deployment. This was caused by an incorrect configuration.

- Initially, Gunicorn could not find the correct module.
- This was resolved by updating the Procfile to match the file and application variable name.

This highlighted an important difference between local execution and cloud deployment: the cloud environment requires explicit configuration for how the application is started.

Testing Approach

Testing was done by:

- Accessing the deployed URL
- Submitting company names through the form
- Verifying that responses were returned correctly
- Checking logs in CloudWatch to confirm requests were being processed

Cloud Engineering Best Practices

Security

- IAM roles were used instead of hardcoded credentials.
- Default security groups controlled access to the EC2 instance.
- No sensitive data or keys were exposed in the code.

Scalability

- Elastic Beanstalk provides built-in support for scaling.
- The application can be extended to use multiple instances if needed.
- The architecture supports load distribution through AWS-managed services.

High Availability

- Elastic Beanstalk environments are designed to recover from instance failures.
- The application could be deployed across multiple availability zones if scaled.

Cost Optimization

- Free-tier eligible resources were used.
- No unnecessary services were provisioned.
- Resources are intended to be terminated after testing to avoid charges.

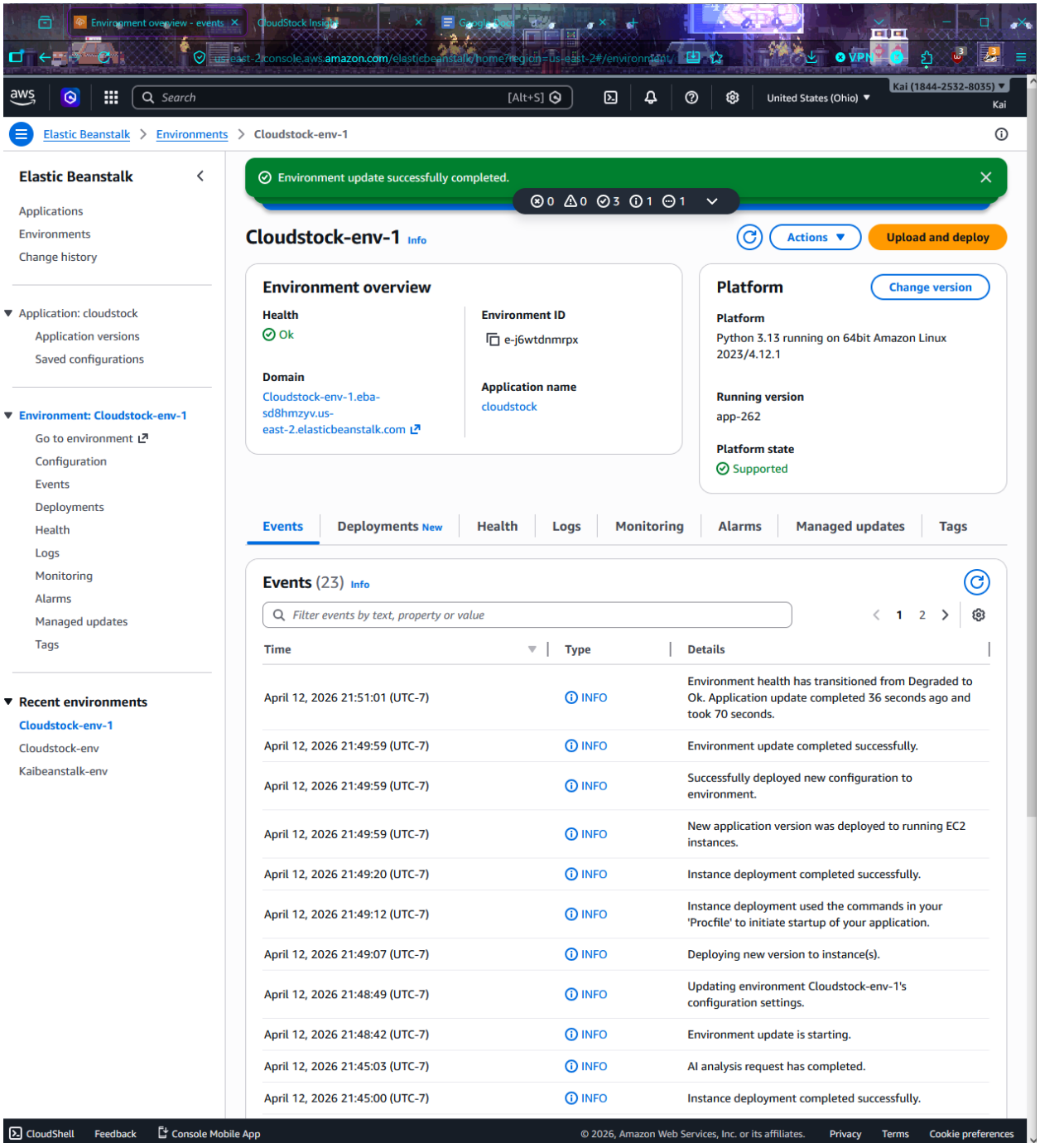
Monitoring

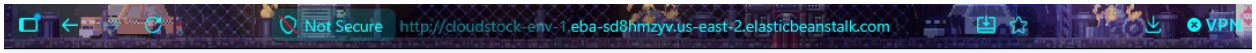
- CloudWatch logs were used to track application activity.
- Logs helped identify and fix deployment issues.
- This provides basic observability into how the system behaves in production.

Lessons Learned & Future Improvements

This project reinforced how different deploying an application to the cloud is compared to running it locally.
One key takeaway was the importance of understanding how the application is started in production. The Procfile and Gunicorn configuration were critical, and small mistakes there caused the entire system to fail.
Another important lesson was the value of logging. CloudWatch made it possible to quickly identify where things were breaking, which made debugging much easier.
If I were to extend this project, I would:

- Replace the mock analysis with a real AI model or API
- Add a database to store user queries and results
- Improve the frontend to make the application more user-friendly
- Add authentication and user accounts
- Implement auto-scaling and load balancing more explicitly


Environment overview - events
CloudStock Insights
Google
us-east-2.console.aws.amazon.com/elasticbeanstalk/home?region=us-east-2#/environment/
VPN
aws
Search
[Alt+S]
United States (Ohio)
Kai (1844-2532-8035)
Kai
Elastic Beanstalk > Environments > Cloudstock-env-1
Elastic Beanstalk
Applications
Environments
Change history
Application: cloudstock
Application versions
Saved configurations
Environment: Cloudstock-env-1
Go to environment
Configuration
Events
Deployments
Health
Logs
Monitoring
Alarms
Managed updates
Tags
Recent environments
Cloudstock-env-1
Cloudstock-env
Kaibeanstalk-env
Environment update successfully completed.
0 0 3 1 1
Cloudstock-env-1 Info
Actions
Upload and deploy
Environment overview
Health
Ok
Environment ID
e-j6wtdnmrpx
Domain
Cloudstock-env-1.eba-sd8hmzyv.us-east-2.elasticbeanstalk.com
Application name
cloudstock
Platform
Change version
Platform
Python 3.13 running on 64bit Amazon Linux 2023/4.12.1
Running version
app-262
Platform state
Supported
Events
Deployments New
Health
Logs
Monitoring
Alarms
Managed updates
Tags
Events (23) Info
Filter events by text, property or value
1 2
Time
Type
Details
April 12, 2026 21:51:01 (UTC-7)
INFO
Environment health has transitioned from Degraded to Ok. Application update completed 36 seconds ago and took 70 seconds.
April 12, 2026 21:49:59 (UTC-7)
INFO
Environment update completed successfully.
April 12, 2026 21:49:59 (UTC-7)
INFO
Successfully deployed new configuration to environment.
April 12, 2026 21:49:59 (UTC-7)
INFO
New application version was deployed to running EC2 instances.
April 12, 2026 21:49:20 (UTC-7)
INFO
Instance deployment completed successfully.
April 12, 2026 21:49:12 (UTC-7)
INFO
Instance deployment used the commands in your 'Procfile' to initiate startup of your application.
April 12, 2026 21:49:07 (UTC-7)
INFO
Deploying new version to instance(s).
April 12, 2026 21:48:49 (UTC-7)
INFO
Updating environment Cloudstock-env-1's configuration settings.
April 12, 2026 21:48:42 (UTC-7)
INFO
Environment update is starting.
April 12, 2026 21:45:03 (UTC-7)
INFO
AI analysis request has completed.
April 12, 2026 21:45:00 (UTC-7)
INFO
Instance deployment completed successfully.
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



CloudStock Insight

Simple AWS-hosted company analysis MVP

Company Name

Amazon

Generate Analysis

Analysis Result

Company

Amazon

Summary

Amazon appears to have consistent customer interest and room for measured growth.

Risk

Economic pressure may lower customer spending in key segments.

Opportunity

Entering adjacent markets may unlock new customer demand.

i-00ebcdc9d33f74c15 (Cloudstock-env-1)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance summary Info

Instance ID
i-00ebcdc9d33f74c15

IPv6 address
–

Hostname type
IP name: ip-172-31-26-100.us-east-2.compute.internal

Answer private resource DNS name
–

Auto-assigned IP address
–

IAM role
No roles attached to instance profile: aws-elasticbeanstalk-ec2-role

IMDSv2
Required

Operator
–

Public IPv4 address
3.140.14.70 | open address

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-26-100.us-east-2.compute.internal

Instance type
t3.micro

VPC ID
vpc-03a27d176e25abffc

Subnet ID
subnet-092e0935605d0436f

Instance ARN
arn:aws:ec2:us-east-2:184425328035:instance/i-00ebcdc9d33f74c15

Private IPv4 addresses
172.31.26.100

Public DNS
ec2-3-140-14-70.us-east-2.compute.amazonaws.com | open address

Elastic IP addresses
3.140.14.70 (Cloudstock-env-1) [Public IP]

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
| Learn more

Auto Scaling Group name
awseb-e-j6wtdnmrpx-stack-AWSEBAutoScalingGroup-Yu5XvIAj03Nm

Managed
false

Instance details Info

AMI ID
ami-0610f5f7b81006a0b

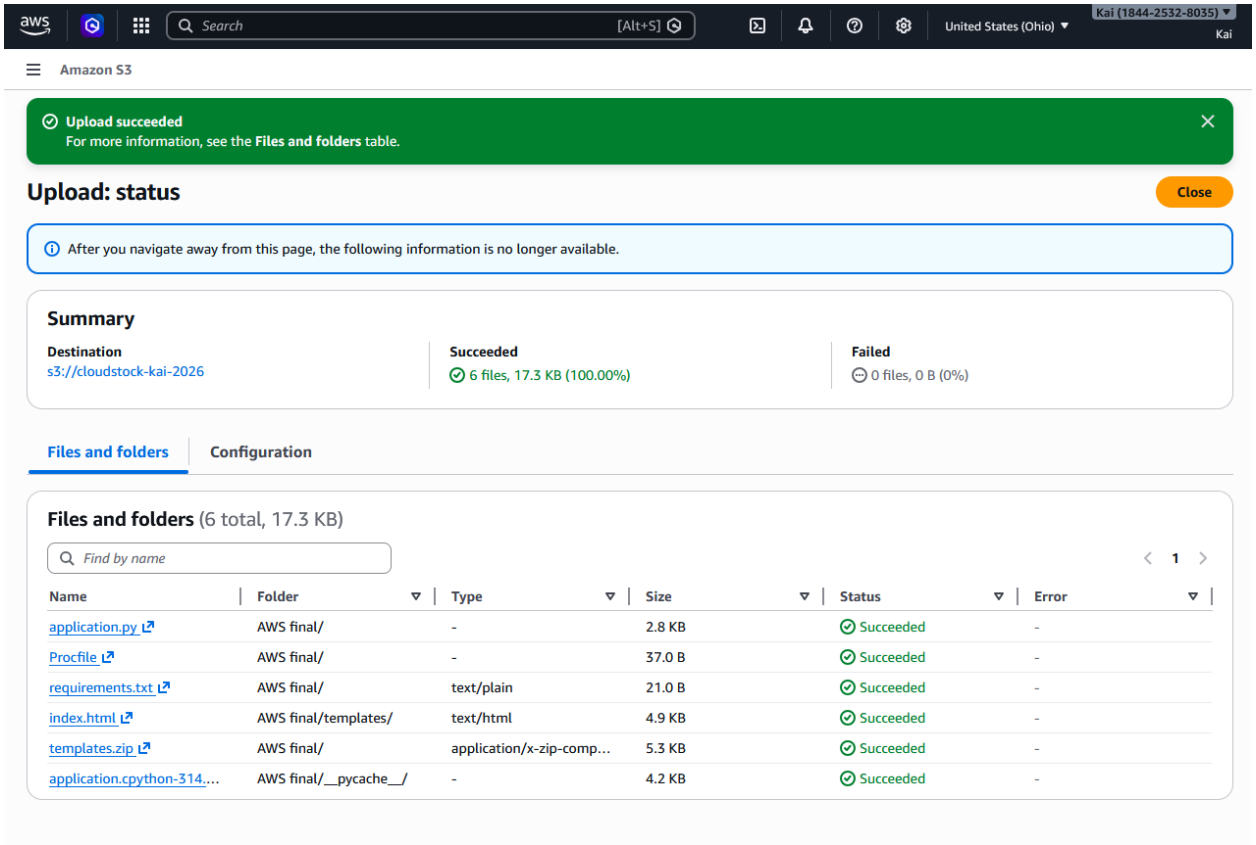
AMI name

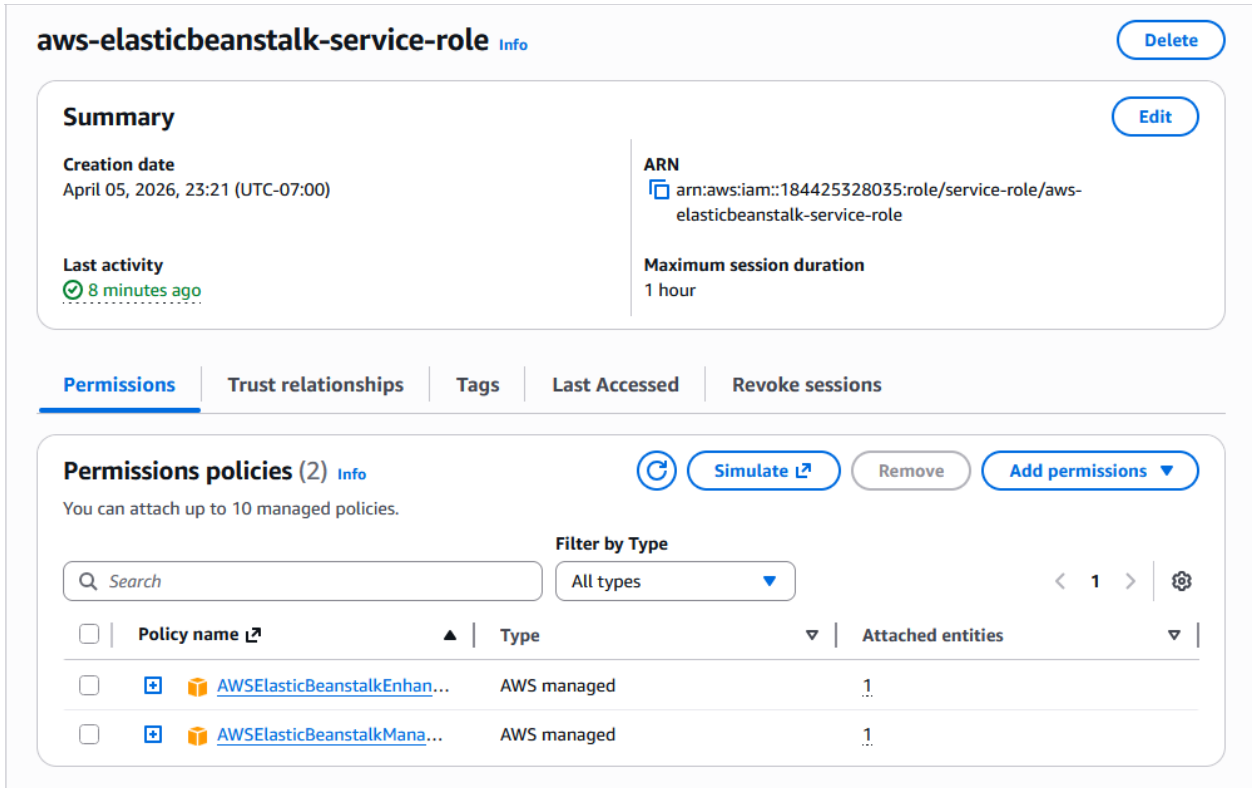
Monitoring
disabled

Allowed image

Platform details
Linux/UNIX

Termination protection


Search [Alt+S]
United States (Ohio)
Kai (1844-2532-8035)
Kai
Amazon S3
Upload succeeded
For more information, see the Files and folders table.
Upload: status
Close
After you navigate away from this page, the following information is no longer available.
Summary
Destination
s3://cloudstock-kai-2026
Succeeded
6 files, 17.3 KB (100.00%)
Failed
0 files, 0 B (0%)
Files and folders
Configuration
Files and folders (6 total, 17.3 KB)
Find by name
Name
Folder
Type
Size
Status
Error
application.py
AWS final/
-
2.8 KB
Succeeded
-
Procfile
AWS final/
-
37.0 B
Succeeded
-
requirements.txt
AWS final/
text/plain
21.0 B
Succeeded
-
index.html
AWS final/templates/
text/html
4.9 KB
Succeeded
-
templates.zip
AWS final/
application/x-zip-comp...
5.3 KB
Succeeded
-
application.cpython-314....
AWS final/__pycache__/
-
4.2 KB
Succeeded
-


aws-elasticbeanstalk-service-role Info
Delete
Summary
Edit
Creation date
April 05, 2026, 23:21 (UTC-07:00)
ARN
arn:aws:iam::184425328035:role/service-role/aws-elasticbeanstalk-service-role
Last activity
8 minutes ago
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (2) Info
Simulate
Remove
Add permissions
You can attach up to 10 managed policies.
Filter by Type
Search
All types
Policy name
Type
Attached entities
AWSElasticBeanstalkEnhan...
AWS managed
1
AWSElasticBeanstalkMana...
AWS managed
1

]

Resource Cleanup & Cost Management

After completing deployment, testing, and documentation, all AWS resources were properly terminated to avoid unnecessary charges. This is an important part of working in cloud environments, since resources continue to incur costs if left running.

Cleanup Steps Performed

1. Elastic Beanstalk Environment
    - Navigated to the Elastic Beanstalk console
    - Selected the active environment
    - Chose “Terminate Environment”
    - This automatically removed:
        - EC2 instance(s)
        - Load balancer (if created)
        - associated security groups
2. Amazon S3
    - Opened the S3 console
    - Emptied the bucket (deleted all objects)
    - Deleted the bucket itself
3. CloudWatch Logs
    - Verified logs were created during testing
    - Left logs as-is (minimal cost), or optionally deleted log groups
4. IAM Roles
    - Default roles created by Elastic Beanstalk were left intact since they do not incur charges
    - No custom roles with elevated permissions were created

Cost Awareness

This project was designed to stay within AWS Free Tier limits:

- Used small instance types
- Limited runtime duration
- Avoided unnecessary services